Android Embarcado e Framework

Aula 6: Segurança e Depuração

Prof. Dr. Eng. Isaías Bittencourt Felzmann isaias@ic.unicamp.br
Monitora: Girlana Souza dos Santos g295743@dac.unicamp.br

ELDORADO INSPIRAÇÃO PELO NOVO

UNICAMP

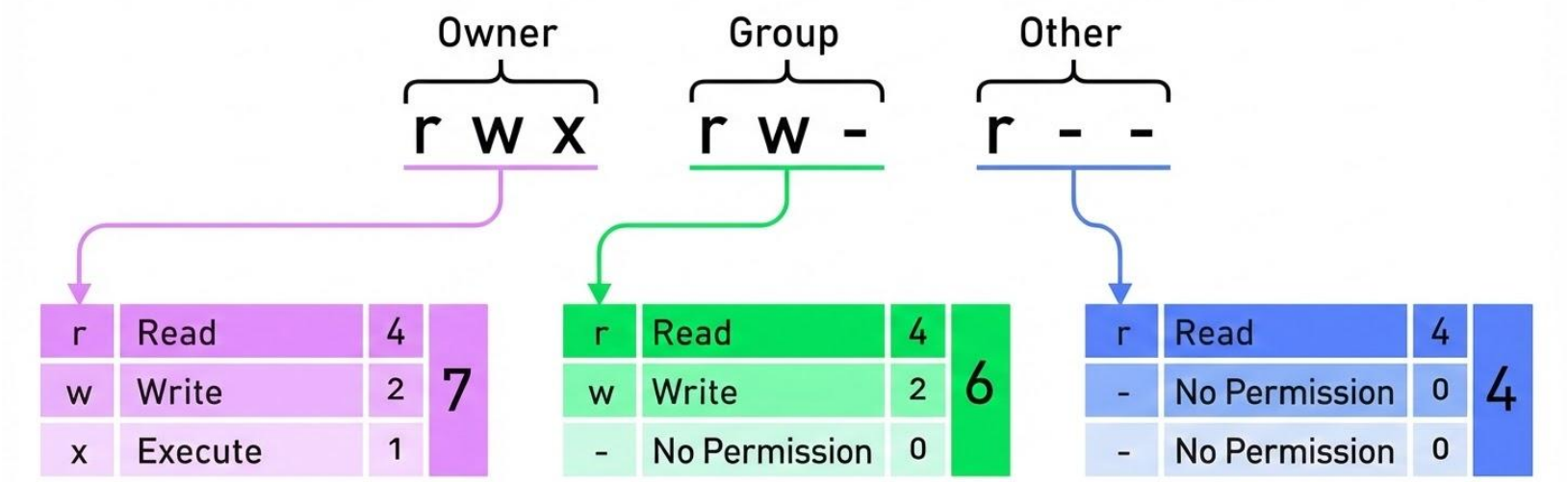
Softex

MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO

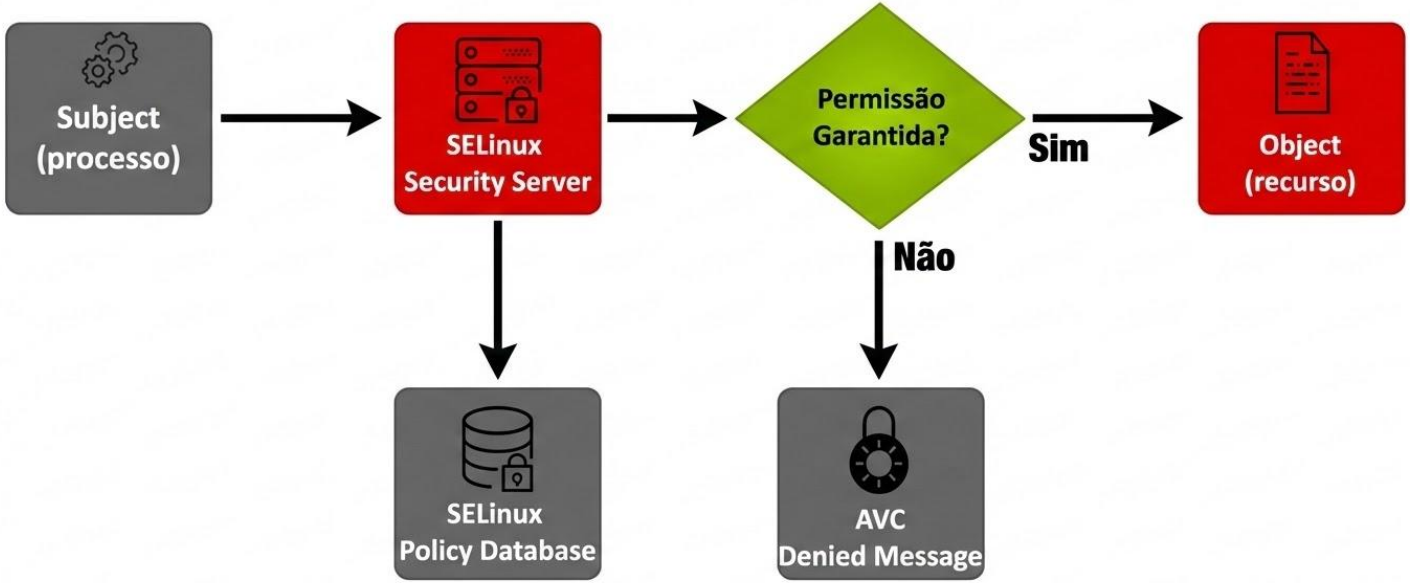
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

Facetas de Segurança

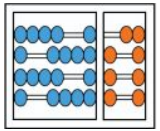
Recapitulando: Modelo de permissões Unix



SELinux: Um Nível Além



Modelo Básico de Segurança Android

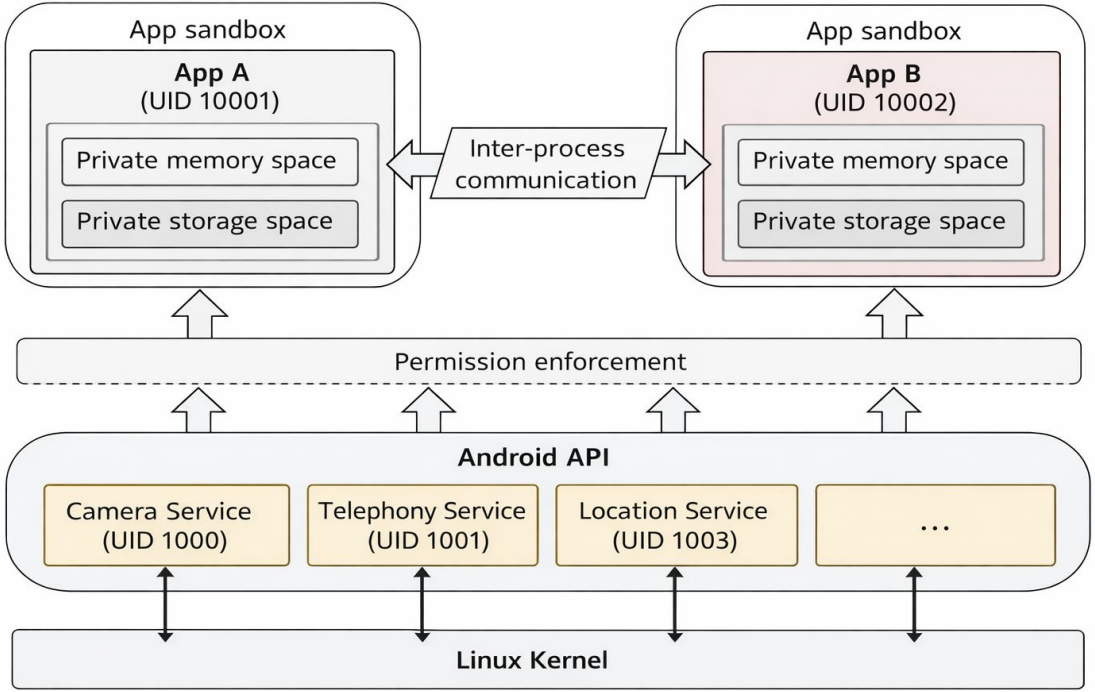


- Cada aplicação executa com um **UID Linux exclusivo**.
  - Recursos da aplicação são isolados por **UID e sistema de arquivos**.

- Aplicações executam em processos isolados.
  - Comunicação entre processos ocorre via Binder (IPC);
  - O Binder transporta a identidade do chamador (UID);
  - Serviços validam permissões antes de acessar recursos.

- Acesso a recursos do sistema é controlado por:
  - permissões Android (framework);
  - políticas SELinux (kernel).

Sandbox Android



Tipos de Permissões Android

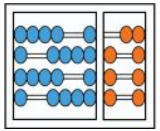
Level 3 Signature or System
- restrito para aplicativos do sistema
- restrito para OS e fabricantes de dispositivos

Level 2 Signature
- apenas para aplicativos de um desenvolvedor (vinculado a um certificado)
- durante o ataque sua usabilidade é restrita apenas para aplicativos com este certificado

Level 1 Dangerous
- deve ser aprovado pelo usuário durante a instalação
- no caso de ataque pode causar vazamento de dados sensíveis

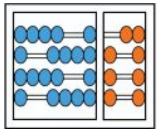
Level 0 Normal
- atribuído ao aplicativo automaticamente
- eles apresentam apenas risco mínimo
- no caso de ataque sua usabilidade é limitada apenas ao aplicativo atacado

Proteções ao Usuário Android



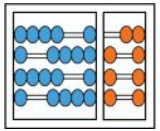
- **Autenticação do Usuário:**
  - Senha / PIN / Padrão
  - Biometria
- **Criptografia de Dados:**
  - FDE (Full Disk Encryption)
  - FBE (File-Based Encryption)
- **Direct Boot:**
  - Parte do sistema funciona antes do desbloqueio
  - Seperação entre Device Encrypted (DE) e Credential Encrypted (CE)
- **Proteção de Chaves:**
  - Keystore / Keymaster;
  - Uso de Trusted Execution Environment.

Customização e limites



- **Permissões Customizadas**
  - Apps podem definir novas permissões.
  - Usadas para proteger APIs e componentes próprios.
- **Limitações e riscos:**
  - Permissões são strings - baixo isolamento por si só.
  - Dependem de assinatura e definição correta.
- **Problemas possíveis:**
  - Apps instalados fora de ordem.
  - Conflitos de definição.
  - Exposição acidental de funcionalidades.
- **Recomendações:**
  - Preferir permissões do sistema.
  - Usar signature-level quando necessário.
  - Evitar expor APIs sensíveis sem controle.

Depuração em Android

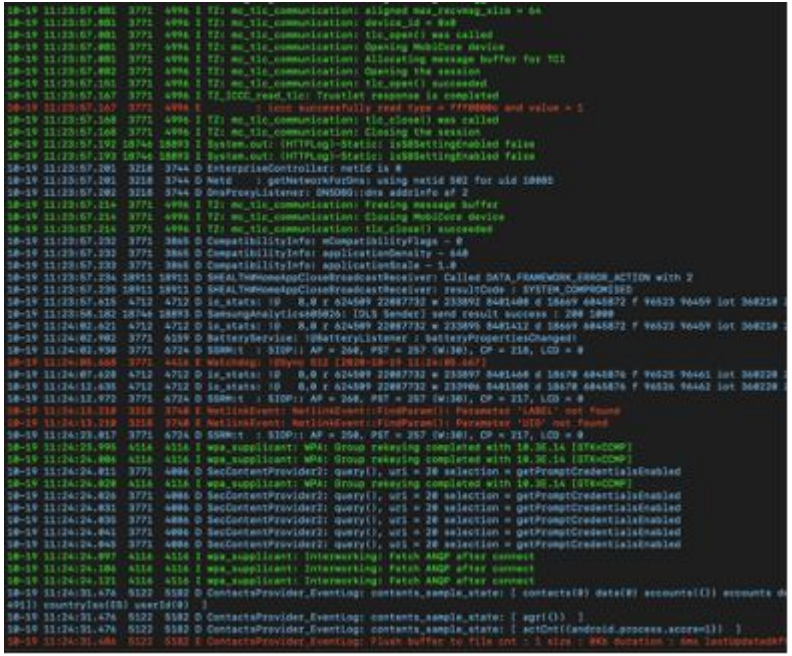


- Android é um sistema complexo, em múltiplas camadas.
  - Vários componentes distribuídos - apps, serviços, IPC, HAL, Kernel.
  - Falhas nem sempre são visíveis, mas podem acontecer em qualquer componente.

- **Principais desafios:**
  - Execução distribuída (processos isolados);
  - Comunicação indireta (Binder);
  - Falhas silenciosas (permissão negada, retorno inválido);
  - Observar estado interno.

- **Abordagem: Observar → Reproduzir → Isolar → Validar → Corrigir.**

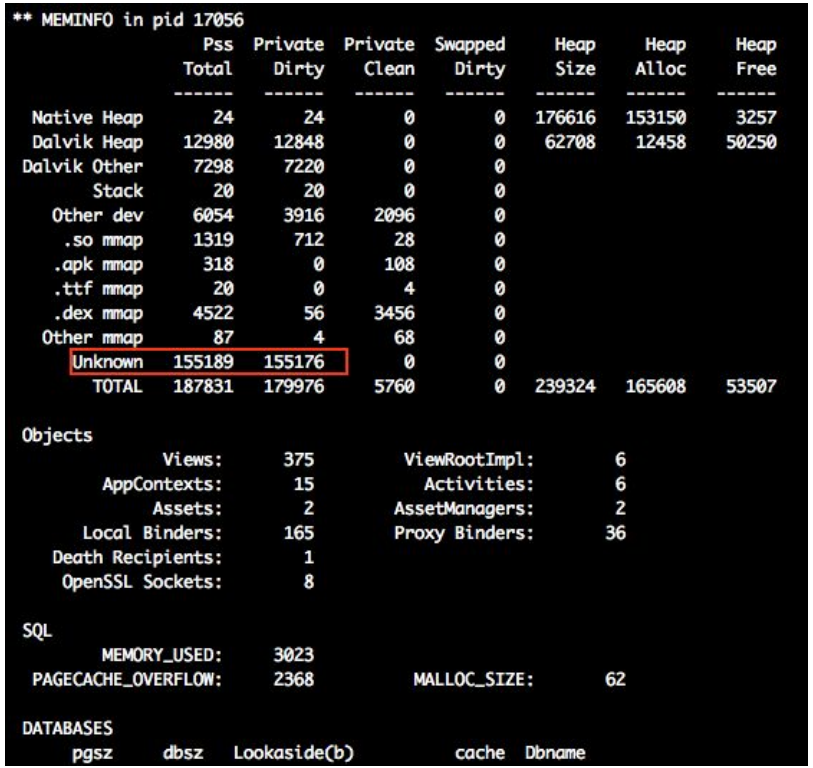
Ferramentas de Depuração



Logcat

- Funcionalidade:
  - Coleta logs de todo o sistema (apps, serviços, sistema).
  - Mostra eventos em ordem temporal.
- Limitações:
  - Não mostra estado interno diretamente;
  - Não garante que tudo foi logado;
  - Não segue execução entre processos automaticamente.

Ferramentas de Depuração

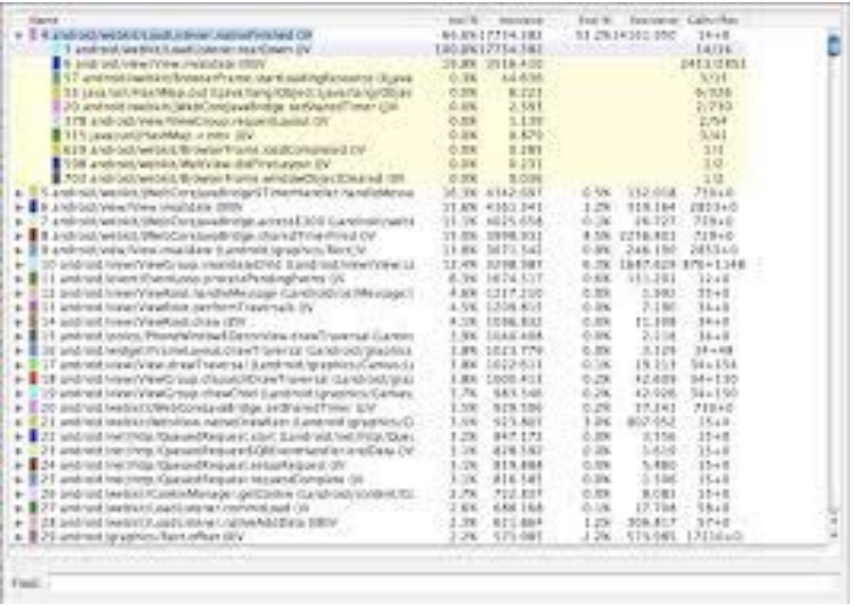


```
** MEMINFO in pid 17056
                   Pss  Private  Private  Swapped     Heap     Heap     Heap
                 Total    Dirty    Clean    Dirty     Size    Alloc     Free
                ------   ------   ------   ------   ------   ------   ------
  Native Heap       24       24        0        0   176616   153150     3257
  Dalvik Heap    12980    12848        0        0    62708    12458    50250
 Dalvik Other     7298     7220        0        0
        Stack       20       20        0        0
    Other dev     6054     3916     2096        0
     .so mmap     1319      712       28        0
    .apk mmap      318        0      108        0
    .ttf mmap       20        0        4        0
    .dex mmap     4522       56     3456        0
   Other mmap       87        4       68        0
      Unknown   155189   155176        0        0
        TOTAL   187831   179976     5760        0   239324   165608    53507

 Objects
               Views:      375         ViewRootImpl:        6
         AppContexts:       15           Activities:        6
              Assets:        2        AssetManagers:        2
       Local Binders:      165        Proxy Binders:       36
    Death Recipients:        1
     OpenSSL Sockets:        8

 SQL
         MEMORY_USED:     3023
  PAGECACHE_OVERFLOW:     2368          MALLOC_SIZE:       62

 DATABASES
      pgsz     dbsz   Lookaside(b)          cache  Dbname
```

Dumpsys

- Funcionalidade:
  - Mostra estado interno de serviços do sistema;
  - Fornece visão estruturada;
  - Permite inspecionar: serviços ativos, atividades, memória, permissões.
- Limitações:
  - Não mostra fluxo de execução;
  - Não mostra histórico de eventos;
  - Pode ser difícil interpretar.

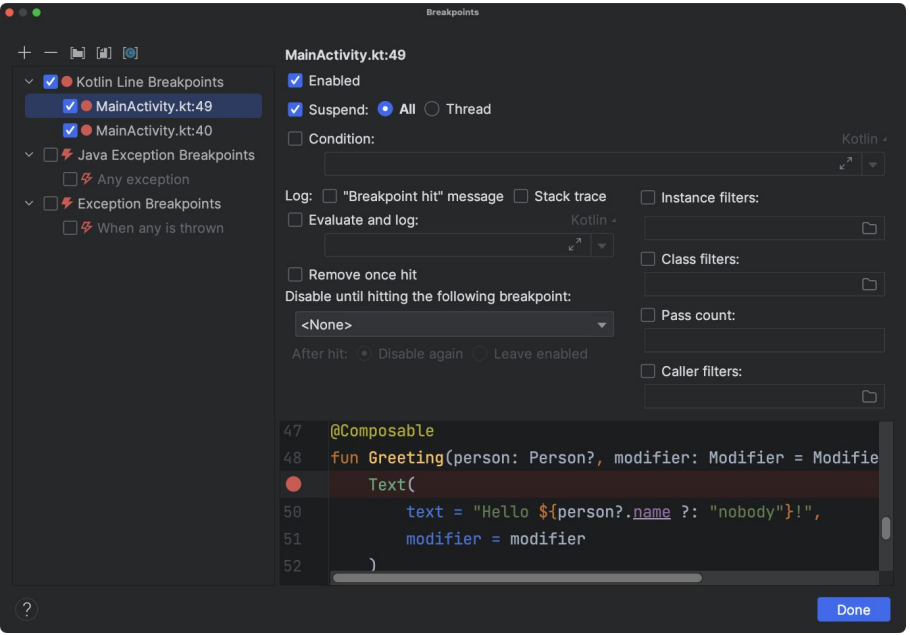
Ferramentas de Depuração



Traceview

- Funcionalidade:
  - Mostra execução dentro de um processo;
  - Perfil de chamadas de métodos;
  - Identifica gargalos e bloqueios.
- Limitações:
  - Não mostra comunicação entre processos;
  - Não mostra estado do sistema;
  - Não ajuda com permissões ou Binder diretamente.

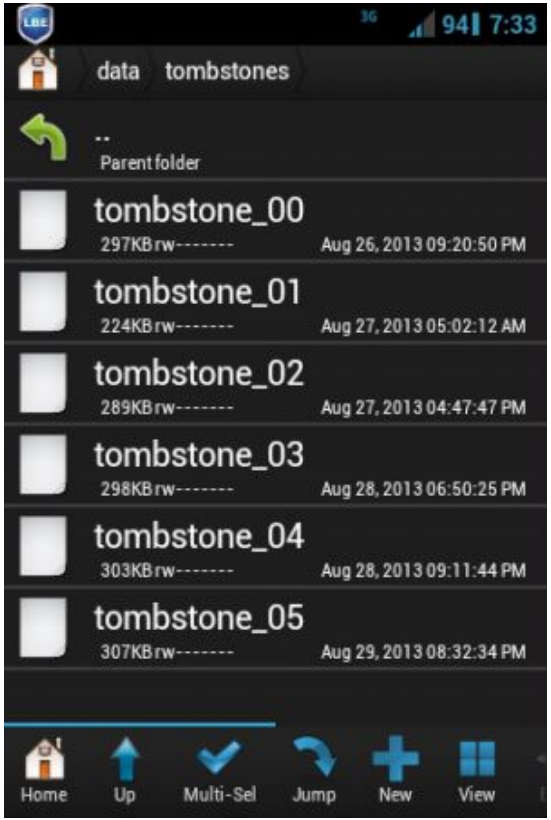
Ferramentas de Depuração



gdb e Android Studio Debugger

- Funcionalidade:
  - Debug de código nativo ou Java;
  - Permite: breakpoints, inspeção de memória, análise detalhada.
- Limitações:
  - Não resolve problemas de arquitetura (Binder, permissões);
  - Difícil de usar em sistema completo.

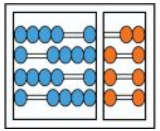
Ferramentas de Depuração



Tombstone

- Funcionalidade:
  - Dump detalhado de falha nativa
  - Mostra: stack trace, registradores, contexto do crash.
- Limitações:
  - Não ajuda se não houve crash;
  - Não mostra fluxo anterior;
  - Não explica causa automaticamente.

Ferramentas de Depuração

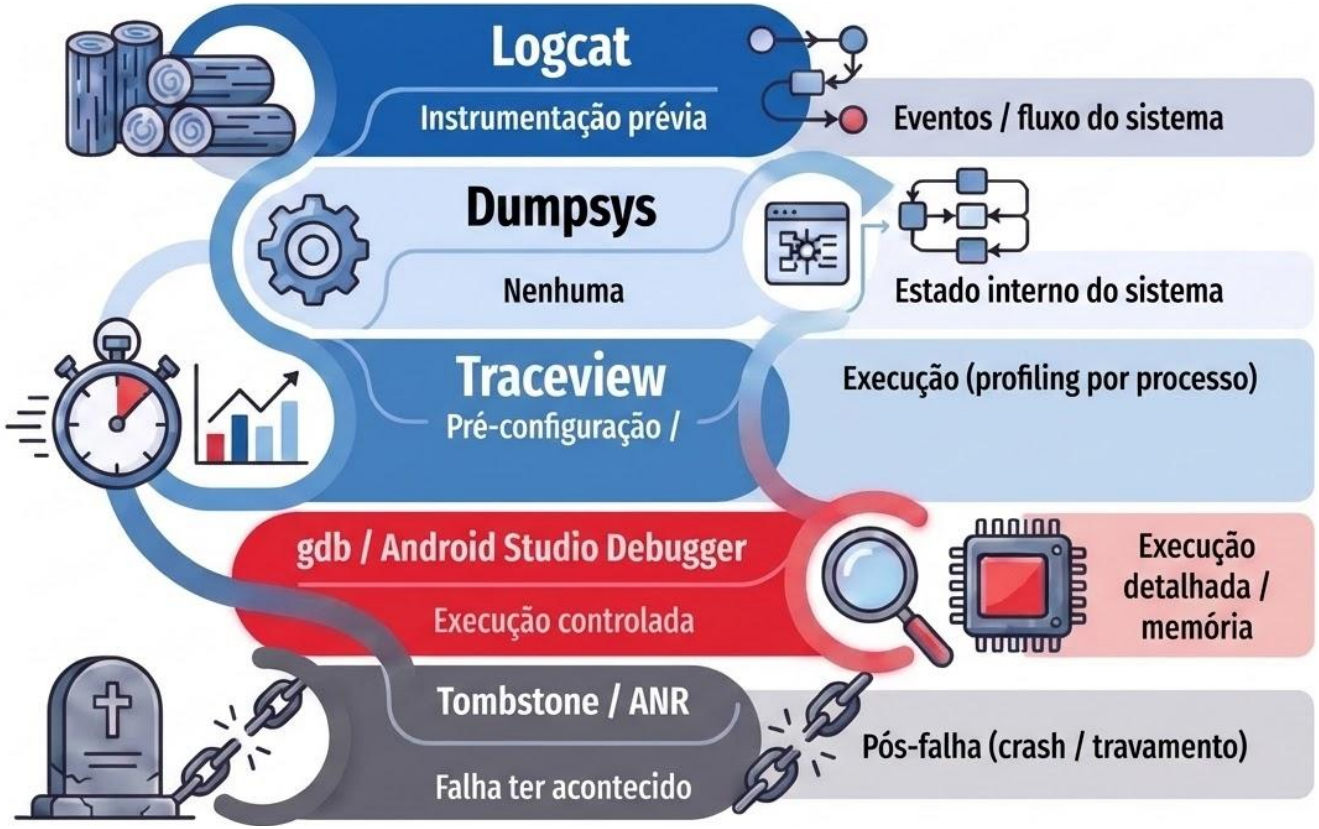


```
"SampleWriterThread" prio=5 tid=19 Sleeping
  | group="main" sCount=1 dsCount=0 flags=1 obj=0x132c0c40 self=0xe0a8fc10
  | sysTid=23427 nice=0 cgrp=top-app sched=0/0 handle=0xbf5f41e0
  | state=S schedstat=( 516243 141394 1 ) utm=0 stm=0 core=2 HZ=100
  | stack=0xbf4f1000-0xbf4f3000 stackSize=1040KB
  | held mutexes=
  at java.lang.Thread.sleep(Native method)
  - sleeping on <0x0271f72c> (a java.lang.Object)
  at java.lang.Thread.sleep(Thread.java:442)
  - locked <0x0271f72c> (a java.lang.Object)
  at java.lang.Thread.sleep(Thread.java:358)
  at com.anr.sample.DataDumup.setNumber(DataDumup.java:37)
  - locked <0x03652b00> (a com.anr.sample.DataDumup)
  at com.anr.sample.App$1.run(App.java:16)
  at java.lang.Thread.run(Thread.java:923)
```

ANR (Application Not Responding)

- Funcionalidade:
  - Detecta travamento da app;
  - Indica: thread bloqueada ou operação longa.
- Limitações:
  - Não explica diretamente a causa;
  - Não mostra histórico completo;
  - Não resolve falhas lógicas.

Resumo de Ferramentas



Expectativa vs Realidade

App (Prática 5)	Na indústria:
UI simples, foco funcional	✓ Arquitetura (MVVM, Clean, etc.)
✓ Chamadas diretas ao serviço	✓ Tratamento de falhas e estados
✓ Sem tratamento robusto de erro	✓ Integração com sistema (lifecycle, background, energia)
✓ Sem preocupação com ciclo de vida completo	✓ UX consistente e responsiva

Service (Prática 4)	Na indústria:
✓ Interface Binder simples	✓ Interface bem definida (AIDL completo)
✓ Lógica linear e direta	✓ Controle de concorrência e sincronização
✓ Sem controle de concorrência	✓ Validação de permissões e segurança
✓ Sem validação rigorosa de permissões	✓ Integração com system_server e politicas do sistema

HAL (Prática 3)	Na indústria:
✓ Implementação direta (sem padrão formal)	✓ Uso de HIDL/AIDL HAL (interface padronizada)
✓ Comunicação simplificada com hardware	✓ Comunicação robusta (I2C, SPI, etc.)
✓ Sem tratamento robusto de erro	✓ Tratamento de falhas e retries
✓ Sem versionamento ou compatibilidade	✓ Versionamento e compatibilidade com o framework

Obrigado!